Entangled Enhancement in Modular Autonomous Driving Systems: An Empirical Study of Cascade Effects from Single-Component Retraining

Rashedul Arefin Ifty, Fumio Machida
University of Tsukuba
Tsukuba, Japan
ifty.rashedul@sd.cs.tsukuba.ac.jp, machida@cs.tsukuba.ac.jp

***Abstract*—Machine learning systems in autonomous driving are maintained incrementally: one component is retrained as its domain drifts while the rest of the pipeline stays fixed. We study the downstream effect of such an update in a modular perception–prediction–planning pipeline, built from a YOLOv11n detector (C1), a constant-velocity predictor (C2), and a rule-based Intelligent Driver Model planner (C3). We train C1 on Boston nuScenes scenes and fine-tune it on Singapore scenes, measuring every component in two modes, *isolated* with ground-truth inputs and *pipeline* with live upstream inputs. Rather than a single retraining, we run a campaign of 15 updates varying data size, training length, and seed. Two effects related to the analytically modeled notion of *entangled enhancement* appear: a non-monotone cascade, in which an update improves an interface yet worsens the plan, and a decoupling between a component's aggregate metric and its interface behavior. The disruption is a landscape rather than a constant, with the planned path moving between 1.33 and 7.80 m. A further 12 updates using a class-balanced training split remove the small-data accuracy penalty and yield *strict* entangled enhancement in three cases, where the detector improves on its own metric while the planning output degrades at the same time. Every isolated-mode metric remained bit-identical, confirming that all changes propagated through the component interfaces. Building on these findings we propose a cascade-aware retraining assessment method and evaluate its low-cost interface-drift front-end, which flags risky updates with a real but imperfect signal.**

***Index Terms*—machine learning systems, autonomous driving, retraining, cascade effects, software reliability, modular pipelines, regression testing**

I. Introduction

Autonomous driving systems are built either as a single end-to-end (E2E) network that maps sensor input directly to a driving plan [1], [2], or as a modular pipeline of separately trained perception, prediction, and planning components. In deployment, such systems are maintained under domain drift one component at a time: a perception model is retrained for a new city, a predictor is updated as agent statistics shift, a planner is revised for new regulations. This component-scoped maintenance is only well-posed in a modular pipeline, where components are separable and independently retrainable. In an E2E model the smallest retrainable unit is the entire network, so we adopt the modular pipeline as the setting for this study.

Retraining one component while the rest stay fixed is the common case in practice, and it is also where a subtle hazard arises. This work builds directly on Wang and Machida [3], who model a two-component MLS as a continuous-time Markov chain and name the failure mode we study, *entangled enhancement*: improving an upstream component on its own metric can still degrade the system, because downstream components have learned to operate on the previous upstream output. A follow-on study casts this as an availability–continuity trade-off [4]. Both are stochastic models that need empirical calibration. So far no one has measured how a single-component retraining event actually propagates through a real perception–prediction–planning pipeline, nor how that propagation varies from one retraining to the next. That is the gap this paper addresses.

The missing element is a controlled measurement of this propagation on a real pipeline, and this paper supplies it. We build a three-component pipeline on the nuScenes dataset [5]: C1, a YOLOv11n [6] camera detector, C2, a constant-velocity trajectory predictor, and C3, a rule-based planner from the Intelligent Driver Model (IDM) family [7] whose proposal-and-score design follows PDM-Closed [8]. We train C1 on Boston scenes and fine-tune it on Singapore scenes, a well-known domain gap within nuScenes, keeping C2 and C3 frozen. The experiments center on two components. C1 is the retrained unit, and C2 is its direct consumer. The interface between them, the stream of detected objects, is where the cascade travels. C3 is kept as the system-level readout that reports whether the change helps or harms the final plan.

The main instrument is dual-mode evaluation, and it exists to separate a component's own error from the error it inherits from upstream. Each downstream component is measured twice: in *isolated* mode with ground-truth inputs, which exposes only its own error, and in *pipeline* mode with live upstream outputs, which exposes its own error plus whatever it inherits. The difference between the two modes is the inherited part, so its change across a retraining event measures the cascade that the retraining injects.

A single retraining, however, is one point. To learn whether the cascade is a fixed property or a variable one, we run a *campaign*: 15 retraining updates of C1 that vary the amount of fine-tuning data, the number of training epochs, and the random seed, each measured through the same dual-mode harness. This extends the study from a single observation to a

landscape of retraining outcomes, and it motivates a practical follow-up question: can a low-cost check, computed from the detector's outputs alone, predict which updates disturb the plan the most?

Our measurements yield seven findings, detailed in Section V.

F1. (Architectural.) The component-level retraining question is unobservable in an E2E architecture and unaffordable at E2E training costs; a modular pipeline is the enabling architecture.

F2. (Isolation.) The isolation property that stochastic MLS models assume holds exactly: the isolated-mode metrics of the frozen C2 and C3 stayed bit-identical across every retraining event.

F3. (Cascade existence.) Retraining C1 alone produced large, measurable changes downstream, concentrated at the first C1→C2 interface.

F4. (Non-monotone cascade.) An update can improve C2's pipeline error yet worsen C3's planning error. In our reference run C2 improved by 73.98% while C3 worsened by 4.45%.

F5. (Metric decoupling.) C1's aggregate mAP@50 can *drop* even as its outputs become more useful to C2, so a component's own metric is not a reliable admission criterion.

F6. (A landscape, not a constant.) Across the 15 updates the downstream disruption ranges widely (plan shift 1.33–7.80 m), the planner is hurt by most updates but helped by a few, and the effect grows with fine-tuning data, so no single number summarizes the entanglement.

F7. (Strict entangled enhancement.) Once a class-balanced fine-tune removes the small-data mAP penalty, δ_1 turns positive in 6 of 12 updates, and in 3 of them the planner degrades at the same time. This is the strict condition (2), observed here in a real driving pipeline.

Contributions. This paper contributes (1) a dual-mode measurement method for retraining-induced cascades in modular MLS, with an isolation check; (2) a controlled measurement, to our knowledge the first on a real perception–prediction–planning pipeline, of how single-component retraining under a geographic domain shift propagates downstream, run as a 15-update campaign that reveals a non-monotone cascade, a decoupling between aggregate and interface metrics, and a wide landscape of outcomes; and (3) a cascade-aware retraining assessment method (CARA) built on this measurement, whose interface-drift front-end and admission rule we evaluate on the campaign.

The remainder of the paper is organized as follows. Section II formalizes the system model, including the definition of entangled enhancement, and the dual-mode methodology. Section III presents the proposed CARA method. Section IV details the experimental setup and the retraining campaign. Section V reports the evaluation results, both the empirical entanglement findings and the evaluation of CARA. Section VII reviews related work, and Section VIII concludes.

II. System Model and Methodology

A. Formal Pipeline Model

We model the driving stack as a composition of three functions,

$$S = C_3 \circ C_2 \circ C_1, \qquad S(x) = C_3\big(C_2(C_1(x))\big), \tag{1}$$

where x is a camera keyframe and the three components carry out the canonical perception–prediction–planning decomposition:

- **C1 (Perception):** $z_1 = C_1(x)$ is a set of detected objects, each with a class, a position, and an extent in the ego frame.
- **C2 (Prediction):** $z_2 = C_2(z_1)$ is, for every detected object, a forecast trajectory over the next several seconds.
- **C3 (Planning):** $z_3 = C_3(z_2)$ is the ego vehicle's planned trajectory, a sequence of positions over a 3 s horizon.

The intermediate signals z_1 and z_2 are the pipeline interfaces, and each is a cascade channel: a change in C_1's output distribution shifts z_1, which its direct consumer C_2 then propagates into z_2 and C_3. Fig. 1 shows the pipeline annotated with the before/after numbers from the reference retraining of this study. Only C_1 is retrained (Boston → Singapore); the isolated (*iso*) metrics of the frozen C_2 and C_3 stay constant, while their pipeline (*pipe*) metrics move because their inputs shift.

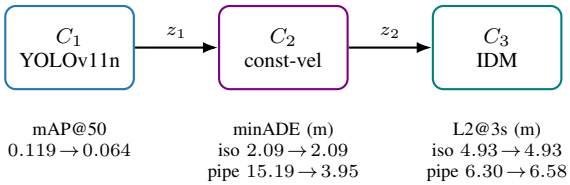

Fig. 1: Pipeline $C_1 \to C_2 \to C_3$ with before/after numbers.

B. Entangled Enhancement

The failure mode this paper investigates is *entangled enhancement*, named by the stochastic MLS model of Wang and Machida [3]. In words, an upstream component is retrained and improves on its own metric, yet the system as a whole degrades, because downstream components were tuned to the statistics of the *previous* upstream output and are disturbed when that output changes. Writing δ_1 for the change in the C1 metric, with positive δ_1 denoting an improvement, and Δ_3 for the change in the terminal (planning) pipeline metric, with positive Δ_3 denoting a regression, the strict condition is

$$\underbrace{\delta_1 > 0}_{\text{C1 improves}} \;\wedge\; \underbrace{\Delta_3 > 0}_{\text{system degrades}} . \tag{2}$$

A closely related but weaker phenomenon is *metric decoupling*: C1's aggregate metric moves in the opposite direction to the usefulness of its outputs to the downstream

consumer, so $\delta_1 < 0$ while the C1→C2 interface metric improves. Decoupling shows that a component's own score is an unreliable proxy for its effect on the system, which is the same lesson as entangled enhancement even when the strict sign condition (2) is not met. As Section V reports, we observe both regimes. Small-data fine-tuning drives $\delta_1 < 0$ across the main campaign, which yields decoupling and non-monotone cascades, while a class-balanced fine-tune lifts δ_1 above zero and produces the strict condition (2) itself.

C. Dual-Mode Evaluation

The core instrument is the measurement of each downstream component in two modes. Let $z_k^\star$ denote the ground-truth value of interface k. In *isolated* mode a component receives ground-truth inputs, so for C_2 and C_3 we evaluate

$$z_2^{\text{iso}} = C_2(z_1^\star), \qquad z_3^{\text{iso}} = C_3(z_2^\star). \tag{3}$$

In *pipeline* mode a component receives the live output of its upstream neighbor,

$$z_2^{\text{pipe}} = C_2(C_1(x)), \qquad z_3^{\text{pipe}} = C_3(C_2(C_1(x))). \tag{4}$$

Isolated mode exposes a component's own error, and pipeline mode exposes its own error plus the error inherited from upstream. For a component C_k with error metric M_k (lower is better), the difference between the two is the inherited part, which we call the *cascade gap* at C_k,

$$\Gamma_k = M_k^{\text{pipeline}} - M_k^{\text{isolated}}. \tag{5}$$

A retraining event R applied to an upstream component changes the pipeline metrics. If the system is properly modular, it leaves all isolated metrics of untouched components unchanged. We call this the *isolation property*:

$$M_k^{\text{isolated}}(\text{before } R) = M_k^{\text{isolated}}(\text{after } R) \quad \forall \text{ frozen } C_k. \tag{6}$$

Equation (6) is not assumed but treated as a testable property of the apparatus: any leakage of the retraining event into a frozen component's isolated metric indicates contamination, and we verify the equation bit-exactly in Section V. The cascade *injected* by R at component C_k is then

$$\Delta_k = M_k^{\text{pipeline}}(\text{after } R) - M_k^{\text{pipeline}}(\text{before } R), \tag{7}$$

which, given (6), equals the change in the cascade gap Γ_k.

D. Cascade Diagnostic and Coupling Factor

The canonical cascade hypothesis is a conjunction of three conditions: the retrained upstream component improves on its own metric, the frozen terminal component is stable in isolation, and it worsens in the pipeline. Using δ_1 and Δ_3 from Section II-B and (7), we encode the diagnostic as

$$\textsc{cascade} \equiv \big(\delta_1 > 0\big) \ \wedge \ \big(\Delta_3^{\text{iso}} = 0\big) \ \wedge \ \big(\Delta_3 > \epsilon\big), \tag{8}$$

with ϵ a small noise floor. When (8) holds, the coupling from the retrained component to the system output is summarized by the *cascade coupling factor*

$$\rho_{1\to 3} = \frac{\Delta_3}{\delta_1}. \tag{9}$$

On a synthetic pipeline with a known planted cascade, (8)–(9) recover $\rho_{1\to 3} \approx 0.84$, which serves as an apparatus check prior to the empirical runs. As Section V shows, the empirical campaign does not satisfy the conjunction (8) in its canonical form, because $\delta_1 < 0$ throughout while Δ_3 varies in sign, so $\rho_{1\to 3}$ is not cleanly measurable and must be read together with the full cascade vector rather than as an isolated scalar.

E. Single-Component Retraining Under Domain Shift

Each retraining event follows the same procedure, and only C1 is ever changed:

1) Train C1 on the source domain (Boston scenes). Freeze C2 and C3 permanently.
2) Measure the five-quantity profile on the target-domain validation set (Singapore): C1 detection quality (mAP@50), C2-isolated minADE, C2-pipeline minADE, C3-isolated L2@$\{1,2,3\}$s, C3-pipeline L2@$\{1,2,3\}$s.
3) Fine-tune C1 on the target domain (Singapore scenes), producing the retrained C1$'$.
4) Re-measure the identical five-quantity profile with C1$'$ in place.
5) Verify the isolation property (6) and compute the injected cascades (7) at C2 and C3.

The Boston-to-Singapore shift within nuScenes is a genuine and well-studied domain gap: left- versus right-hand traffic, different vehicle and agent mixes, and different lighting conditions [5].

F. Metrics

C1 is scored with standard detection metrics (mAP@50, mAP@50-95, precision, recall) on the target-domain validation images. For C_2, the metric is the minimum average displacement error (minADE). For an agent with forecast positions $\hat{p}_t^{(m)}$ under mode m and ground-truth positions $p_t^\star$ over T future steps,

$$\text{minADE} = \min_m \ \frac{1}{T}\sum_{t=1}^{T} \big\|\hat{p}_t^{(m)} - p_t^\star\big\|_2, \tag{10}$$

averaged over agents. Our deterministic predictor has a single mode, so the $\min$ collapses to that mode. For C_3, the metric is the L2 displacement error between the planned ego trajectory $\hat{e}_t$ and the human driver's recorded trajectory $e_t^\star$ at horizon τ,

$$\text{L2@}\tau = \big\|\hat{e}_\tau - e_\tau^\star\big\|_2, \qquad \tau \in \{1,2,3\}\ \text{s}, \tag{11}$$

reported together with collision rate. We also report, for the campaign, a *plan shift*: the distance between the plan produced with the incumbent C1 and the plan produced with the retrained C1 on the same scene, which measures how far a retraining moves the driving decision independent of the human path. The L2 metric is the standard nuScenes open-loop planning protocol used by the E2E literature [2], [9], [10], with the differential-use caveat discussed in Sections V and VII.

III. Cascade-Aware Retraining Assessment

The measurement method of Section II exposes a concrete hazard for maintenance: a retrained component that regresses on its own metric may still improve its neighbor while degrading the system, so neither component-level nor first-interface validation is a sufficient admission criterion. We turn the method into an admission check for component updates, Cascade-Aware Retraining Assessment (CARA). CARA formalizes, as an explicit gate, the procedure we later run across the campaign of Section IV. Fig. 2 summarizes the decision and Algorithm 1 states it. The evaluation of CARA on real updates is deferred to Section V.

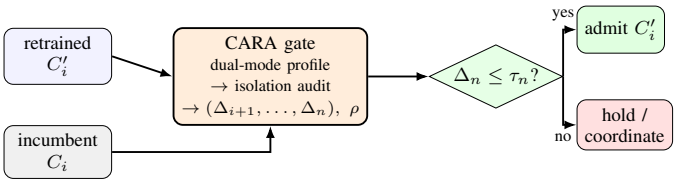

Fig. 2: The CARA admission check for a retrained component.

A. The Admission Procedure

Given a pipeline $C_1 \to \cdots \to C_n$, a frozen audit set A, and a candidate update R to component C_i, CARA records the dual-mode profile with the incumbent C_i, swaps in the retrained C_i', and records it again on the identical audit set with fixed randomness. It requires the isolation property (6) to hold for every frozen C_k. Any violation signals leakage or nondeterminism and invalidates the assessment. In our campaign this audit passed bit-exactly (F2). It then computes the injected cascade Δ_k (7) at every downstream component and reports the full vector $(\Delta_{i+1}, \ldots, \Delta_n)$, because adjacent deltas can carry opposite signs, together with the coupling factor $\rho_{i \to n} = \Delta_n / \delta_i$. A negative ρ with an improving δ_i is the entangled-enhancement regime. As our campaign shows, ρ is well-defined only when δ_i is unambiguous. It is therefore not cleanly measurable on the updates of Table II, where $\delta_1 < 0$ throughout, but it is well-posed on the class-balanced updates of Section V-G, whose detector metric moves definitely upward. The update is admitted only if $\Delta_n \le \tau_n$ and safety-critical auxiliaries such as collision rate do not regress. Otherwise it is held or routed to coordinated retraining of the downstream consumers.

B. Interface-Drift Front-End

The full admission procedure runs the entire pipeline on the audit set, which is the cost of a complete evaluation. We precede it with a lower-cost screen that reads only the detector's own outputs. For a candidate detector, the screen compares the statistics of its detections against those of the incumbent on the same audit scenes, using a Kolmogorov–Smirnov statistic over the continuous features (per-image count, confidence, box-center position) and a Jensen–Shannon divergence over the class histogram, aggregated into one interface-drift score in $[0, 1]$. This follows

Algorithm 1: Cascade-Aware Retraining Assessment (CARA)

Input: pipeline $C_1 \to \ldots \to C_n$; audit set A; candidate update R on C_i; tolerance τ_n
Output: admit / hold decision, cascade vector, coupling factor ρ

$P_{\text{before}} \leftarrow \textsc{DualModeProfile}(C_i, A)$
$C_i' \leftarrow R(C_i)$; $P_{\text{after}} \leftarrow \textsc{DualModeProfile}(C_i', A)$
foreach *frozen* C_k, $k > i$ **do**
  if $M_k^{iso}(\textit{after}) \neq M_k^{iso}(\textit{before})$ **then**
    return invalid `// isolation violated (contamination)`
  end
end
$\Delta_k \leftarrow M_k^{\text{pipe}}(\text{after}) - M_k^{\text{pipe}}(\text{before})$ for $k = i{+}1..n$
$\rho \leftarrow \Delta_n / \delta_i$
if $\Delta_n \le \tau_n$ ***and*** *collision rate not worse* **then**
  return admit C_i'
else
  return hold / coordinate downstream retraining
end

the between-stage two-sample tests used in production data-validation systems [11], and it is model-free: it consumes only the detection records the pipeline already emits. The screen identifies which updates warrant the full dual-mode assessment, and its predictive value is evaluated in Section V-H.

C. Why the Method Needs a Modular Architecture

Admission never rests on δ_i, which addresses metric decoupling (F5), and it reports the full delta vector, so an improvement at one interface cannot mask a regression further downstream (F4). The isolation audit makes silent contamination detectable by exact equality testing, a check available only because the architecture is modular (F1). The whole method is therefore constructible only in the modular setting. In an E2E monolith there are no frozen components, isolated modes, or interfaces at which Δ_k could be measured. Its cost is inference on a fixed audit set, with no training.

IV. Experimental Setup

A. Dataset and Components

We use nuScenes-mini v1.0 [5]. Partitioning by recorded location and splitting each domain into train and validation gives `boston_train`/`boston_val` (2/2 scenes) and `singapore_train`/`singapore_val` (3/3 scenes); all cascade measurements use `singapore_val` ($n = 3$). For C1 training, the 3D box annotations are projected onto the front-camera keyframes to produce 2D labels over eight driving classes.

C1 is YOLOv11n [6], a camera detector rather than a LiDAR one such as CenterPoint [12], so absolute magnitudes will differ with a richer perception front-end even though the

cascade quantities are defined independently of it. It is fine-tuned for 10 epochs on `boston_train` from COCO weights to give the incumbent detector, then fine-tuned further on `singapore_train` starting from the Boston checkpoint to give each retrained detector. At inference, each 2D detection is lifted to an ego-frame ground position by intersecting its bottom-center pixel ray with the ground plane, supplying the agent positions z_1 that C2 consumes. C2 is a deterministic, parameter-free constant-velocity predictor: in isolated mode it rolls agents out at their ground-truth velocity, and in pipeline mode it anchors a zero-velocity rollout at C1's single-frame detections, so that any change in its pipeline output is attributable solely to a change in C1. C3 is a rule-based Intelligent Driver Model [7] proposal-and-score planner that selects, from a speed-aware grid of candidate trajectories $\mathcal{P}$, the one minimizing progress cost plus a soft proximity penalty against the predicted agent futures z_2,

$$\pi^\star = \arg\min_{\pi\in\mathcal{P}} \; J_{\text{prog}}(\pi) \; + \; \lambda \sum_{a\in z_2} \phi\big(d(\pi, a)\big), \qquad (12)$$

where $d(\pi, a)$ is the closest approach between candidate π and agent a, ϕ is a soft penalty on proximity, and λ weights safety against progress. This reproduces the PDM-Closed [8] architecture without its learned scorer; the dependence of (12) on z_2 is the channel through which perturbed detections reach the plan.

B. The Retraining Campaign

A single fine-tuning event is one point in a large space of possible updates. To map the space rather than sample it once, we run a campaign of 15 **retraining updates** of C1, each measured through the identical dual-mode harness of Section II-E. The campaign varies three knobs:

- **Fine-tuning data size:** the full `singapore_train` set (119 images), half of it (60), and a quarter (30). Smaller sets are the harder, more data-starved cases.
- **Training length:** 5, 10, and 20 epochs, to test whether longer fine-tuning changes the downstream effect.
- **Random seed:** each setting is repeated with three seeds, so a result can be told apart from seed noise.

The matrix is three data sizes at 10 epochs and two extra training lengths (5 and 20) at the full data size, each over three seeds, giving $3 \times 3 + 2 \times 3 = 15$ updates. The incumbent C1 (Boston-trained) and the frozen C2 and C3 are shared across all 15 updates, and only the Singapore fine-tune of C1 changes between them. Every randomness source is fixed within each update so that the isolation property (6) can be checked for exact equality rather than statistical closeness.

C. Measurement Protocol

The five-quantity profile records one C1 metric together with the isolated and pipeline metrics of C2 and C3, so that each component's own error and its inherited error are separated (Section II-E). For each of the 15 updates the harness records this profile on `singapore_val` for both the incumbent and the retrained C1, then computes δ_1, Δ_2, Δ_3, the plan shift, and the regime label (Section II-B). Because C2 and C3 have no trainable parameters, the isolation check is exact.

V. Evaluation Results

This section reports two evaluations. The first is the empirical study of the cascade itself: one update in full detail as a reference case (Sections V-B–V-E), then the campaign that places it in a landscape (Section V-F). The second evaluates the proposed CARA method on those same updates, through its interface-drift front-end as a predictor of downstream risk (Section V-H) and its admission rule against the measured ground truth (Section V-I).

A. Reference Update: Headline Measurements

The reference update is the full-data, 10-epoch, seed-1 fine-tune. Table I reports its complete before/after profile on `singapore_val`, where "before" is the Boston-trained C_1 and "after" is the Singapore-fine-tuned C_1, and Fig. 3 plots it. The detection quality of C_1 (mAP@50) drops after fine-tuning, C2-pipeline minADE improves sharply (15.19 → 3.95 m), and C3-pipeline L2@3s worsens (6.30 → 6.58 m). Both isolated metrics are unchanged, which is the visual signature of the isolation property (6).

TABLE I: Reference update: measurements on Singapore validation ($n = 3$ scenes).

Metric	Before	After	Δ	Δ%
C1 mAP@50	0.1185	0.0644	−0.0541	−45.66
C1 mAP@50-95	0.0571	0.0258	−0.0313	−54.81
C1 precision	0.6845	0.6709	−0.0136	−1.99
C1 recall	0.0866	0.0477	−0.0389	−44.92
C2 iso. minADE (m)	2.0899	2.0899	0.0000	0.00
C2 pipe. minADE (m)	**15.1930**	**3.9535**	**−11.2395**	**−73.98**
C3 iso. L2@1s (m)	0.6900	0.6900	0.0000	0.00
C3 iso. L2@2s (m)	2.4944	2.4944	0.0000	0.00
C3 iso. L2@3s (m)	4.9289	4.9289	0.0000	0.00
C3 pipe. L2@1s (m)	1.0396	1.0838	+0.0442	+4.25
C3 pipe. L2@2s (m)	3.5017	3.6070	+0.1053	+3.01
C3 pipe. L2@3s (m)	**6.2969**	**6.5770**	**+0.2801**	**+4.45**
C3 pipe. collision rate	0.000	0.000	0.0000	—

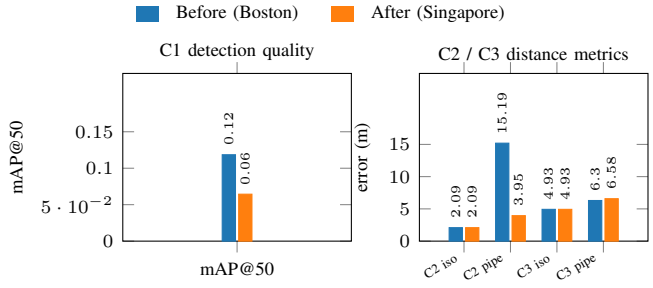

Fig. 3: Reference update on `singapore_val`, before versus after: (a) C1 detection quality, (b) C2/C3 distance metrics.

B. Verification of the Isolation Property (F2)

Before reading any cascade delta, the frozen components must be shown to be untouched, which is a validity check on the apparatus and not itself a result. The three isolated-mode rows are exactly zero-delta. C2-isolated minADE is 2.0899 m in both runs, and C3-isolated L2 is $0.6900/2.4944/4.9289$ m at the three horizons in both runs, to every recorded digit. For deterministic, parameter-free C2 and C3 fed identical ground-truth inputs, this invariance is expected by construction, and any nonzero delta would have signalled state or data leakage between the runs. Its role is to certify that the pipeline-mode deltas in Table I arise solely from the C1 retraining event, transmitted through the component interfaces. This exact-equality check held across all 15 updates in the campaign.

C. Cascade Concentration at the C1→C2 Interface (F3)

The largest single effect in Table I occurs at the C1→C2 interface. With the Boston-trained C1, C2's pipeline-mode minADE was 15.1930 m against an isolated-mode floor of 2.0899 m, a cascade gap Γ_2 of 13.10 m. This Γ_2 conflates two sources: C1 detection error and the loss of velocity information, since C2-isolated uses ground-truth velocity whereas C2-pipeline uses a zero-velocity rollout (Section IV); it therefore bounds the combined effect, not perception error alone. After the Singapore fine-tune, C2-pipeline minADE fell to 3.9535 m ($\Gamma_2 = 1.86$ m), a 73.98% improvement without modifying C2. The change $\Delta_2 = -11.24$ m is attributable to C1 alone, since the velocity-information gap is constant across the two runs, so the first downstream interface dominates the retraining-induced cascade.

D. Non-Monotone Cascade at the System Output (F4)

The first inversion is at the system output. Even though C2-pipeline improved by 73.98%, C3-pipeline worsened at every horizon: $+4.25\%$ at 1 s, $+3.01\%$ at 2 s, and $+4.45\%$ at 3 s (6.2969 m → 6.5770 m, $\Delta_3 = +0.2801$ m). The cascade gap at the planner grew from $\Gamma_3 = 1.37$ m to 1.65 m, with collision rate zero in both runs. The two pipeline-mode deltas therefore carry opposite signs: the retraining event that improved the planner's inputs degraded its output. Read against the diagnostic (8), the run satisfies the isolation clause ($\Delta_3^{\text{iso}} = 0$) and the degradation clause ($\Delta_3 = +0.28$ m $> \epsilon$) but *not* the upstream-improvement clause, because aggregate mAP fell (F5). The run is therefore not a clean instance of entangled enhancement in the strict sense of the base paper [3], which requires $\delta_1 > 0$. It is instead the related non-monotone regime, in which C2's interface improves while the system output degrades. It is nonetheless a controlled, quantified instance of the correction-cascade pattern that the technical-debt literature describes only qualitatively [13]. Fig. 4 illustrates it: C3-isolated remains fixed at 4.93 m, since the planner receives identical ground-truth inputs in both runs, while C3-pipeline drifts from 6.30 to 6.58 m.

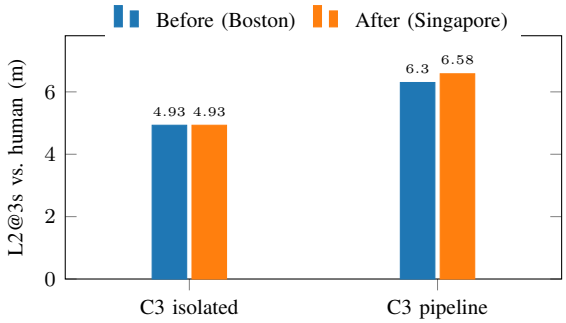

Fig. 4: Cascade signature on planning: isolated fixed, pipeline drifts.

E. Aggregate Metric Decoupling at C1 (F5)

Table I exhibits a second inversion, this one between C1's own score and the usefulness of its outputs. The Singapore-fine-tuned C1, the checkpoint that improved C2 by 74%, scores *lower* on its own aggregate validation metric than its Boston-trained predecessor (mAP@50 $0.1185 \rightarrow 0.0644$, -45.66%; recall $0.0866 \rightarrow 0.0477$). The immediate cause is small-data overfitting: the Singapore fine-tune set contains only 119 images (944 instances) with a sparse class mix, and the fine-tuned detector converges to detection regimes that diverge from the validation label distribution, as evident in the confusion matrix of Fig. 5, where most non-car classes collapse into the background row. The cascade, however, does not propagate through the aggregate score but through the detection outputs themselves: a detector can lose mAP while emitting outputs that are more useful to its downstream consumer on the relevant scenes. Aggregate component metrics and interface behavior therefore decouple, here moving in opposite directions with large magnitudes on both sides, so a component's own metric is not a reliable admission criterion. This is the per-component, system-scale analogue of the negative-flip phenomenon reported for classifier updates [14].

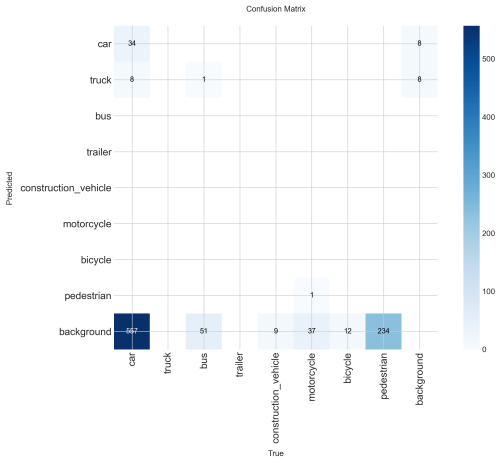

Fig. 5: Confusion matrix of the Singapore-fine-tuned C_1.

F. The Campaign: A Landscape, Not a Constant (F6)

The reference update is a single point in this space. Table II reports all 15 updates of the campaign (Section IV-B), and three patterns emerge.

First, the cascade is always present, but its magnitude varies substantially. The plan shift, defined as the distance the planned path moves when the incumbent C1 is replaced by the retrained one, ranges from 1.33 m to 7.80 m. The same form of retraining alters the driving decision only slightly in one campaign and substantially in another, so no single number summarizes the entanglement.

Second, the direction of the terminal effect is campaign-dependent. In the reference update the planner regressed, but across the 15 updates $\Delta_3 > 0$ (planner worse) in 11 and $\Delta_3 < 0$ in 4. Two of the latter are material improvements of 0.20 and 1.04 m, both from small-data (quarter) updates, while the other two are negligible at under a millimetre and are labelled benign in Table II. The reference conclusion that retraining degrades the planner therefore does not hold universally.

Third, the amount of fine-tuning data governs the magnitude. Averaging the 10-epoch updates by data size, the mean plan shift is 6.12 m for the full set, 2.98 m for half, and 3.46 m for a quarter, so the full-data updates produce the largest and most variable downstream change. A detector that adapts more strongly to the new city induces a larger change throughout the pipeline. Across all 15 updates $\delta_1 < 0$ (Table II), which is the small-data effect of F5, so this campaign realizes the decoupling and non-monotone regimes repeatedly (4 updates match the reference decoupling pattern of $\delta_1 < 0$ with $\Delta_2 < 0$, and 2 are essentially benign) but not the strict entangled enhancement of (2), which requires $\delta_1 > 0$. Section V-G removes exactly that obstacle.

TABLE II: The 15-update retraining campaign on `singapore_val`. δ_1: detector-score change (positive better); Δ_2: prediction-error change (negative better); Δ_3: planner-error change (negative better); shift: plan shift in metres.

Update	img	δ_1	Δ_2	Δ_3	shift	regime
Full, 10 ep, s1	119	−0.022	+5.39	+0.68	7.80	other
Full, 10 ep, s2	119	−0.022	−1.02	+1.34	7.40	decoupling
Full, 10 ep, s3	119	−0.025	−2.23	+0.26	3.17	decoupling
Half, 10 ep, s1	60	−0.053	+2.38	+0.26	3.17	other
Half, 10 ep, s2	60	−0.044	+1.63	+0.26	3.17	other
Half, 10 ep, s3	60	−0.045	+2.17	+0.24	2.59	other
Quarter, 10 ep, s1	30	−0.060	+1.83	+0.26	3.17	other
Quarter, 10 ep, s2	30	−0.057	−5.29	−0.20	3.29	decoupling
Quarter, 10 ep, s3	30	−0.066	−3.85	−1.04	3.93	decoupling
Full, 5 ep, s1	119	−0.037	+2.32	+0.24	2.59	other
Full, 5 ep, s2	119	−0.052	+0.83	−0.00	1.33	benign
Full, 5 ep, s3	119	−0.052	+0.83	−0.00	1.33	benign
Full, 20 ep, s1	119	−0.033	+8.00	+0.24	2.59	other
Full, 20 ep, s2	119	−0.006	+9.38	+0.26	3.17	other
Full, 20 ep, s3	119	−0.017	+2.42	+0.26	3.17	other

G. Strict Entangled Enhancement Under Class-Balanced Retraining (F7)

Every update in Table II lost detector accuracy, and the cause is the class mix rather than the cascade: the 119-image Singapore set is dominated by `car`, so a small fine-tune overfits that class and the aggregate mAP falls. Because the strict condition (2) requires $\delta_1 > 0$, this small-data artifact, and not the pipeline, is what stood between the campaign and a textbook instance of entangled enhancement.

We therefore repeat the retraining with a class-balanced training split. Images containing rare classes are oversampled so that the per-class instance distribution flattens, with a repeat factor $(n_{\max}/n_c)^p$ per class c clipped to $[1, 12]$. The validation split is left untouched, so mAP stays comparable to every other run in this paper. At p=0.5 the split grows from 119 to 399 images and the max/min class ratio falls from 34.8 to 17.7. We run two balance strengths ($p \in \{0.5, 1.0\}$), two training lengths (10 and 20 epochs), and three seeds, giving 12 updates.

The balancing achieves its purpose. The detector now *improves* on its own metric in 6 of the 12 updates, reversing the sign of δ_1 that held throughout Table II. More importantly, three of those updates also degrade the planner, which is exactly the strict entangled-enhancement condition (2): C1 gets better on its own terms while the system it feeds gets worse. Table III lists them together with a representative counter-example. All three arise at 20 epochs, under both balance strengths and across two seeds, so the effect is not a single lucky run. Longer adaptation is what lifts C1 past the incumbent, and it is precisely in that regime that the downstream cost appears.

TABLE III: Class-balanced updates. The first three satisfy the strict condition (2): $\delta_1 > 0$ (C1 improves) with $\Delta_3 > 0$ (system degrades). Full matrix: $\delta_1 > 0$ in $6/12$, strict EE in $3/12$.

Update	δ_1	Δ_2	Δ_3	shift	strict EE
p=0.5, 20 ep, s1	+0.031	+3.87	+0.24	2.59	**yes**
p=1.0, 20 ep, s3	+0.021	+8.59	+0.24	2.59	**yes**
p=1.0, 20 ep, s2	+0.012	+0.88	+0.24	1.26	**yes**
p=1.0, 10 ep, s2	+0.002	−7.70	−0.63	4.67	no

This is the paper's central empirical claim. Entangled enhancement is not only an analytically modeled hazard [3]; it occurs in a real perception–prediction–planning pipeline, and it is reachable by an ordinary maintenance action, namely retraining one component on new-city data with a balanced class mix. It also sharpens the practical warning: the three strict cases would each have passed a component-level review, because C1's own metric improved, yet each made the driving output worse.

H. Evaluation of CARA's Drift Screen (F6 continued)

The campaign enables an evaluation of the interface-drift front-end of CARA (Section III-B) as a predictor of risk. For each of the 15 updates we compute the drift score of the

retrained detector against the incumbent on the audit scenes and assess how well it anticipates the downstream disruption that only the full pipeline reveals.

Ranked against the plan shift each update caused, the drift score is a real but loose predictor. The rank correlation between drift and plan shift is ≈ 0.41, and Fig. 6 shows the relationship. The agreement is clearest at the extremes: the two updates with the largest plan shifts (7.80 and 7.40 m) also carry the two highest drift scores, and drift follows the same data-size trend as plan shift. The front-end therefore carries usable signal about which updates disturb the plan, although it is far from a precise predictor and is best used to prioritize updates for the full assessment rather than as a standalone decision.

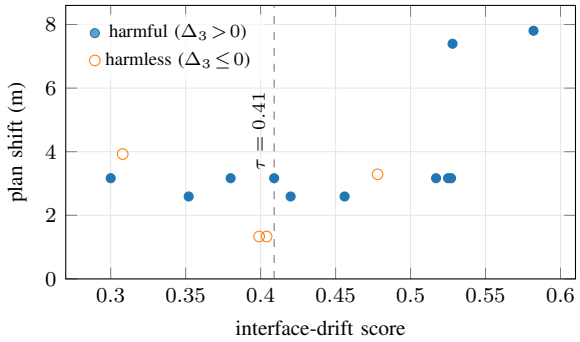

Fig. 6: Interface drift vs. plan shift over the 15 updates (Spearman ≈ 0.41); dashed line is the balanced threshold.

I. Evaluation of CARA's Admission Rule

We next evaluate the drift score as the binary admit/hold rule of CARA's front-end, scored against ground truth. We label an update "harmful" when it actually worsened the planner ($\Delta_3 > 0$); of the 15 updates, 11 are harmful and 4 are not. Treated as a ranking, the drift score places a harmful update above a harmless one about 70% of the time (AUROC ≈ 0.70), consistent with the loose rank correlation above.

As a thresholded decision the rule is sensitive to the threshold, as Table IV shows. At a strict threshold every one of the 15 updates drifts enough to be held, so no harmful update is admitted (recall 1.0) but no update is admitted at all (specificity 0), which is safe but provides no discrimination. Relaxing the threshold to a balanced operating point yields a genuinely selective rule: it catches roughly three-quarters of the harmful updates (recall 0.73) while correctly admitting three-quarters of the harmless ones (specificity 0.75), at precision 0.89. None of the 15 retrained pipelines produced a collision, so the collision-rate-aware clause of Algorithm 1 could not be exercised on this data, and the terminal-L2 clause carries the decision alone.

We interpret these results conservatively. CARA's low-cost front-end works in principle on our data but is not a validated rule, because the discriminating threshold was chosen on the same 15 points against which it is judged, with no held-out set, and with only three audit scenes the evaluation constitutes a proof of concept. The full admission procedure of Algorithm 1, which runs the complete dual-mode profile rather than the drift proxy, is the more reliable decision, and the front-end serves as its low-cost first filter. We report the front-end evaluation because it is what the 15-update campaign can support at this scale.

TABLE IV: CARA drift-rule decisions against ground truth ($\Delta_3 > 0$ = harmful). Confusion counts are (held-harmful, held-harmless, admitted-harmful, admitted-harmless).

Threshold	TP/FP/FN/TN	Rec.	Spec.	Prec.	F1
Strict (τ=0.25)	11/4/0/0	1.00	0.00	0.73	0.85
Balanced (τ=0.41)	8/1/3/3	0.73	0.75	0.89	0.80

J. Training Diagnostics

Both fine-tunes converged within their epoch budgets with monotone loss decay, as Fig. 7 shows for the Boston run: the box, classification, and distribution-focal losses decrease on both the train and validation splits, while precision, recall, and mAP@50 increase, with mAP@50 rising from 0.039 at epoch 1 to 0.119 at epoch 10. Each fine-tune completes in under a GPU-minute, several orders of magnitude below the cost of a single E2E training run [15], which is what renders a campaign of repeated retraining feasible. The cascade propagates through the ego-frame positions of the detected boxes, which alter the forecasts and hence the plan, independently of whether the detections differ visibly to the eye.

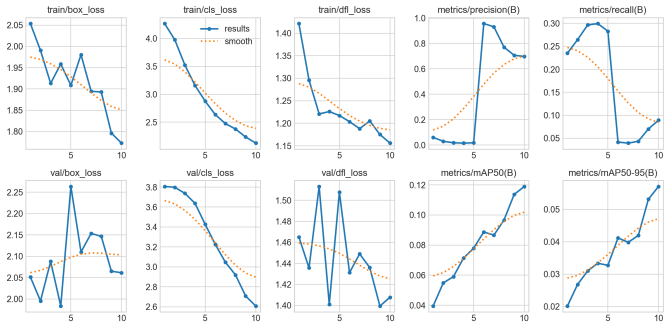

Fig. 7: YOLOv11n C_1 Boston fine-tune training curves.

VI. Discussion

We explain the mechanism behind the non-monotone cascade, place CARA among related maintenance techniques, and state the threats to validity.

A. Why the Cascade Is Non-Monotone

Under the Boston-trained C1, the planner's inputs were substantially displaced ($\Gamma_2 = 13.10$ m) in a regime whose errors the proposal-and-score rules absorbed: missed or grossly misplaced agents leave the proximity penalty inactive, so the planner follows its kinematic prior, which on these scenes approximated the human trajectory. The fine-tuned C1 relocated detections near their true positions and re-activated the penalty with correctly anchored but coarse (zero-velocity)

forecasts, causing the planner to react to obstacles it had previously ignored. Downstream components can thus absorb upstream error, and, symmetrically, upstream improvement can expose masked downstream sensitivity. A retrained component therefore cannot be admitted on its own interface alone.

This mechanism also cautions against a literal interpretation of the terminal metric. Collision rate was zero before and after in every update, and the higher L2 of the reference case arises because the improved detector caused the planner to react to real agents rather than follow a kinematic prior that happened to track the recorded human path. This is the ego-forecasting shortcut of nuScenes open-loop L2, under which perception-agnostic prediction is competitive [10] and open-loop error is weakly tied to closed-loop quality [16]. A regression on this metric may therefore mark a *safer* planner that scores worse on a measure rewarding ignored perception, so the terminal readings here should be read as reduced trajectory agreement, not an established safety regression, and a closed-loop or NAVSIM-style metric [17] is the appropriate safeguard.

B. CARA Among Related Maintenance Techniques

The findings give three maintenance rules that CARA (Section III) enforces directly. An update must not be admitted on the retrained component's own metric, because of the decoupling in F5 and, most directly, the strict cases of F7 in which C1 improved while the system degraded. Admission must not stop at the first interface, because the non-monotone cascade of F4 lets an improvement at C2 coincide with a regression at C3. And because the downstream effect is a landscape (F6) rather than a constant, a maintenance policy should assess several candidate updates rather than trust one.

CARA is one of several complementary techniques for these rules, not a replacement for them. Regression-constrained retraining such as positive-congruent training [14] and backward-compatible training [18] *reduces* the cascade at its source by constraining the update. Uncertainty-propagating interfaces [19] *absorb* the residual by passing distributions instead of point estimates across the perception–prediction boundary. Between-stage data validation [11] *monitors* the interface in production. CARA adds the missing *admission* step that measures the downstream cascade directly before an update is deployed, and its low-cost interface-drift front-end (Section III-B) can operate alongside the same production monitors.

VII. Related Work

A. End-to-End Driving and Its Maintenance Limits

UniAD put detection, tracking, mapping, motion forecasting, occupancy prediction, and planning into a single planning-oriented network [2]. VAD replaced dense rasterized scene representations with vectorized ones for efficiency [9]. SparseDrive introduced fully sparse scene representations and reported the resource profile of the flagship systems: $7.2\times$ faster training and $5\times$ faster inference than UniAD, whose training takes about 144 GPU-hours [15]. The main survey of the field lists interpretability, causal confusion, and robustness under distribution shift as open challenges, and it does not treat component-level maintenance or retraining policy at all [1].

A recent E2E repair literature addresses maintenance directly. CorrectAD builds a self-correction loop that diagnoses failures, generates targeted training video with a diffusion model, and fine-tunes the entire E2E planner on old plus new data, correcting 62.5% of failure cases on nuScenes [20]. R2SE avoids full-model retraining by freezing the generalist model and training reinforcement-learned low-rank adapter specialists on failure regions, motivated by the catastrophic forgetting that naive fine-tuning causes [21]. Both reintroduce modular structure to make maintenance tractable, motivating the modular architecture adopted in this work.

B. Evaluation of Driving Systems

Our planning metric is open-loop displacement error on nuScenes, and the limits of this protocol are well documented. Codevilla et al. showed as early as 2018 that offline prediction error correlates only weakly with actual closed-loop driving quality [16]. Zhai et al. found that a trivial MLP using only ego state, with no perception at all, matches perception-based E2E methods on nuScenes open-loop L2 [10]. Dauner et al. showed that open-loop ego-forecasting and closed-loop driving are misaligned tasks, and that a largely rule-based planner, PDM-Closed, outperformed learned planners [8]. Their later NAVSIM benchmark provides short-horizon non-reactive metrics that align far better with closed-loop performance [17]. These critiques target open-loop metrics used to *rank* architectures; our use is differential (the same frozen planner, same scenes, one upstream change), so much of that weakness cancels, and we report collision rate alongside L2 and revisit the caveat in Section VI.

C. Model Updates, Regression, and Interface Compatibility

What we measure at system scale has per-sample analogues in the model-update literature. Yan et al. showed that model updates cause negative flips, samples the old model handled correctly that the new model gets wrong, at rates of 4–5% even between equal-accuracy retrains of identical architectures, and they proposed positive-congruent training with focal distillation to suppress them [14]. The backward-compatible training of Shen et al. constrains a new upstream representation model so that unchanged downstream consumers keep working [18], which is the same situation as updating C1 beneath a frozen C2 and C3. On the monitoring side, the data validation system of Breck et al. runs two-sample distribution tests between pipeline stages in production [11]. On a nuScenes detector–tracker–predictor stack nearly the same shape as ours, Ivanovic et al. showed that propagating state uncertainty across the perception–prediction interface, instead of passing point estimates, improves forecasting by a wide margin [19]. These works provide mitigation techniques. This paper contributes the complementary controlled measurement of the cascade that such techniques are intended to mitigate.

D. Reliability Modeling of Machine Learning Systems

The technical-debt analysis of Sculley et al. introduced CACE, unstable data dependencies, and correction cascades as qualitative failure modes of ML systems [13]. The base paper for this work, Wang and Machida [3], formalized imperfect retraining of a two-component MLS as a continuous-time Markov chain, named entangled enhancement, and compared progressive and conservative maintenance policies. Their follow-on work casts the same problem as an availability–continuity trade-off [4]. Both are analytical models whose state space grows as 2^N for N components, and both are calibrated on synthetic transition rates. This paper provides an empirical counterpart: the isolation property we verify (F2) is the property their state factorization assumes, and the coupling factor we propose (Section III) is a measured statistic that can replace those synthetic rates.

VIII. Conclusion

We presented a controlled study of single-component retraining in a modular perception–prediction–planning pipeline under a geographic domain shift. Using dual-mode evaluation with a verified isolation control, and running a campaign of 15 retraining updates rather than a single event, we found that retraining C1 alone reliably disturbs the downstream plan, but by an amount that is not a constant. The plan shift ranges from 1.33 to 7.80 m, the planner is hurt by most updates yet helped by a few, and the effect grows with fine-tuning data. In the reference update C1's aggregate metric moved opposite to its interface usefulness, and the same decoupling and non-monotone regimes recur across the campaign. A further 12 updates with a class-balanced training split then delivered the effect in its strict form: with the small-data accuracy penalty removed, δ_1 turned positive in half of the updates, and in three of them the planner degraded at the same time. Entangled enhancement is therefore not only an analytically modeled hazard but an observable one in a real driving pipeline. Together these results show that component-level validation is insufficient for safe MLS maintenance, since each of those three updates would have passed a review of C1 alone. We proposed the CARA admission method for maintaining such pipelines and evaluated its interface-drift front-end, which carries a real but imperfect signal (AUROC ≈ 0.70) and is best used to prioritize updates rather than to decide on them. The broader contribution is methodological: cascade effects can be measured under exact controls, but only in an architecture that exposes its interfaces.

References

[1] L. Chen, P. Wu, K. Chitta, B. Jaeger, A. Geiger, and H. Li, "End-to-end autonomous driving: Challenges and frontiers," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2024.

[2] Y. Hu, J. Yang, L. Chen, K. Li, C. Sima, X. Zhu, S. Chai, S. Du, T. Lin, W. Wang, L. Lu, X. Jia, Q. Liu, J. Dai, Y. Qiao, and H. Li, "Planning-oriented autonomous driving," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.

[3] Z. Wang and F. Machida, "Maintaining performance of a machine learning system against imperfect retraining," in *2024 IEEE 48th Annual Computers, Software, and Applications Conference (COMPSAC)*. IEEE, 2024, pp. 1782–1787.

[4] Z. Wang and F. Machida, "Exploiting the availability-continuity trade-off in imperfect retraining of machine learning systems," in *2025 IEEE 36th International Symposium on Software Reliability Engineering (ISSRE)*, 2025, pp. 406–417.

[5] H. Caesar, V. Bankiti, A. H. Lang, S. Vora, V. E. Liong, Q. Xu, A. Krishnan, Y. Pan, G. Baldan, and O. Beijbom, "nuscenes: A multimodal dataset for autonomous driving," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 11 621–11 631.

[6] G. Jocher and J. Qiu, "Ultralytics yolo11," https://github.com/ultralytics/ultralytics, 2024.

[7] M. Treiber, A. Hennecke, and D. Helbing, "Congested traffic states in empirical observations and microscopic simulations," *Physical Review E*, vol. 62, no. 2, pp. 1805–1824, 2000.

[8] D. Dauner, M. Hallgarten, A. Geiger, and K. Chitta, "Parting with misconceptions about learning-based vehicle motion planning," in *Conference on Robot Learning (CoRL)*, 2023.

[9] B. Jiang, S. Chen, Q. Xu, B. Liao, J. Chen, H. Zhou, Q. Zhang, W. Liu, C. Huang, and X. Wang, "Vad: Vectorized scene representation for efficient autonomous driving," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.

[10] J.-T. Zhai, Z. Feng, J. Du, Y. Mao, J.-J. Liu, Z. Tan, Y. Zhang, X. Ye, and J. Wang, "Rethinking the open-loop evaluation of end-to-end autonomous driving in nuscenes," *arXiv preprint arXiv:2305.10430*, 2023.

[11] E. Breck, N. Polyzotis, S. Roy, S. E. Whang, and M. Zinkevich, "Data validation for machine learning," in *Proceedings of Machine Learning and Systems (SysML/MLSys)*, vol. 1, 2019.

[12] T. Yin, X. Zhou, and P. Krähenbühl, "Center-based 3d object detection and tracking," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021, pp. 11 784–11 793.

[13] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison, "Hidden technical debt in machine learning systems," in *Advances in Neural Information Processing Systems (NIPS)*, vol. 28, 2015, pp. 2503–2511.

[14] S. Yan, Y. Xiong, K. Kundu, S. Yang, S. Deng, M. Wang, W. Xia, and S. Soatto, "Positive-congruent training: Towards regression-free model updates," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021, pp. 14 299–14 308.

[15] W. Sun, X. Lin, Y. Shi, C. Zhang, H. Wu, and S. Zheng, "Sparsedrive: End-to-end autonomous driving via sparse scene representation," *arXiv preprint arXiv:2405.19620*, 2024.

[16] F. Codevilla, A. M. López, V. Koltun, and A. Dosovitskiy, "On offline evaluation of vision-based driving models," in *European Conference on Computer Vision (ECCV)*, 2018.

[17] D. Dauner, M. Hallgarten, T. Li, X. Weng, Z. Huang, Z. Yang, H. Li, I. Gilitschenski, B. Ivanovic, M. Pavone, A. Geiger, and K. Chitta, "Navsim: Data-driven non-reactive autonomous vehicle simulation and benchmarking," in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024.

[18] Y. Shen, Y. Xiong, W. Xia, and S. Soatto, "Towards backward-compatible representation learning," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 6368–6377.

[19] B. Ivanovic, Y. Lin, S. Shrivastava, P. Chakravarty, and M. Pavone, "Propagating state uncertainty through trajectory forecasting," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2022.

[20] E. Ma, L. Zhou, T. Tang, J. Zhang, J. Jiang, Z. Zhang, D. Han, K. Zhan, X. Zhang, X. Lang, H. Sun, X. Zhou, D. Lin, and K. Yu, "Correctad: A self-correcting agentic system to improve end-to-end planning in autonomous driving," *arXiv preprint arXiv:2511.13297*, 2025.

[21] H. Liu, T. Li, H. Yang, L. Chen, C. Wang, K. Guo, H. Tian, H. Li, H. Li, and C. Lv, "Reinforced refinement with self-aware expansion for end-to-end autonomous driving," *arXiv preprint arXiv:2506.09800*, 2025.